

Assessment-1

Girija G

23BAI1403

Experiment-1

AIM

Implementing Deterministic Finite Automaton (DFA) from regular grammar using C language.

Input: The set of states (Q), set of input alphabet (T), set of final states (F) and the set of transitions (Delta) and a terminal string (w).

Output: Transition table and the sequence of transition to accept w.

ALGORITHM

Implementing DFA using C

Step 1: Read the number of states and input symbols of the DFA.

Step 2: Read the input alphabet symbols and store them.

Step 3: Initialize the transition table with invalid values.

Step 4: Read the number of transitions and store state transitions for each input symbol.

Step 5: Display the transition table of the DFA.

Step 6: Read the number of final states and store them.

Step 7: Read an input string starting from the initial state q0.

Step 8: For each symbol in the string, move to the next state using the transition table.

Step 9: If an invalid transition occurs, reject the string immediately.

Step 10: After processing the string, check whether the current state is a final state.

Step 11: Accept the string if the final state is reached; otherwise reject it.

SOURCE CODE

```
#include<stdio.h>
#include<string.h>

int main(){
    int ns,na,nt,nf;
    int i,j;

    printf("Enter number of states:");
    scanf("%d",&ns);

    printf("Enter number of input symbols:");
    scanf("%d",&na);

    char alpha[na];
    printf("Enter input symbols:\n");
    for(i=0;i<na;i++){
```

```

    scanf(" %c",&alpha[i]);
}

int trans[ns][na];
for(i=0;i<ns;i++){
    for(j=0;j<na;j++){
        trans[i][j]=-1;
    }
}

char table[ns][ns];
for(i=0;i<ns;i++){
    for(j=0;j<ns;j++){
        table[i][j]='-';
    }
}

printf("Enter number of transitions:");
scanf("%d",&nt);

int curr,next;
char sym;
printf("Enter transitions(curr input next):\n");
for(i=0;i<nt;i++){
    scanf("%d %c %d",&curr,&sym,&next);
    for(j=0;j<na;j++){
        if(alpha[j]==sym){
            trans[curr][j]=next;
            table[curr][next]=sym;
            break;
        }
    }
}

printf("\nTransition Table:\n");
printf(" ");
for(i=0;i<ns;i++){
    printf("q%d ",i);
}
printf("\n");

for(i=0;i<ns;i++){
    printf("q%d ",i);
    for(j=0;j<ns;j++){
        printf(" %c ",table[i][j]);
    }
    printf("\n");
}

printf("Enter number of final states:");
scanf("%d",&nf);

```

```

int final[nf];
printf("Enter final states:\n");
for(i=0;i<nf;i++){
    scanf("%d",&final[i]);
}

int a=1;
while(a==1){
    printf("do you want to enter a string 1/0\n");
    scanf("%d",&a);
    if(a==0) break;

    char s[100];
    printf("Enter input string:");
    scanf("%s",s);

    int state=0,found,invalid=0;
    printf("(q0)");

    for(i=0;i<strlen(s);i++){
        found=-1;
        for(j=0;j<na;j++){
            if(alpha[j]==s[i]){
                found=j;
                break;
            }
        }

        if(found==-1||trans[state][found]==-1){
            printf(",%c)->invalid\n",s[i]);
            printf("String rejected\n");
            invalid=1;
            break;
        }

        state=trans[state][found];
        printf(",%c)->(q%d",s[i],state);
    }

    if(invalid) continue;

    printf("\n");

    int acc=0;
    for(i=0;i<nf;i++){
        if(state==final[i]){
            acc=1;
            break;
        }
    }

    if(acc)

```

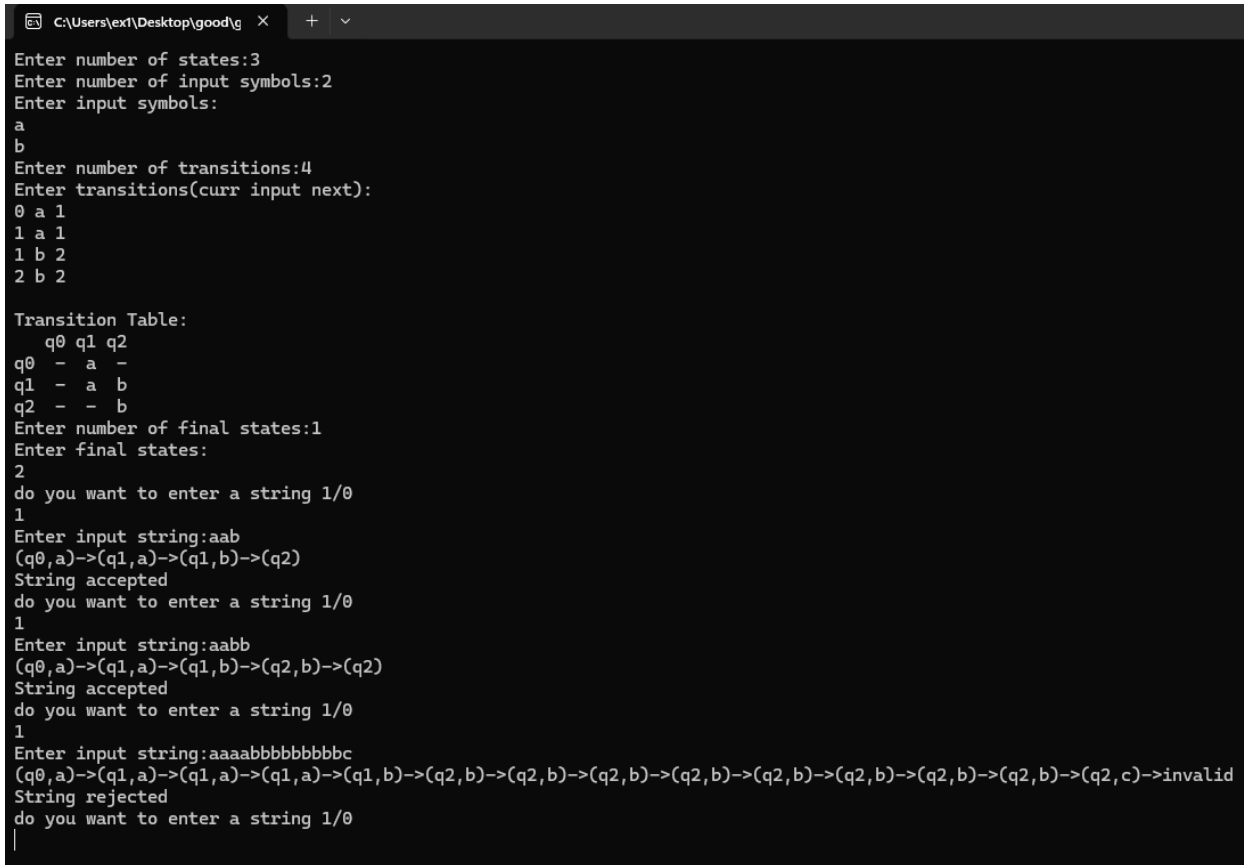
```

        printf("String accepted\n");
    else
        printf("String rejected\n");
}

return 0;
}

```

OUTPUT



```

C:\Users\ext\Desktop\goodg x + v
Enter number of states:3
Enter number of input symbols:2
Enter input symbols:
a
b
Enter number of transitions:4
Enter transitions(curr input next):
0 a 1
1 a 1
1 b 2
2 b 2

Transition Table:
  q0 q1 q2
q0 - a -
q1 - a b
q2 - - b
Enter number of final states:1
Enter final states:
2
do you want to enter a string 1/0
1
Enter input string:aab
(q0,a)->(q1,a)->(q1,b)->(q2)
String accepted
do you want to enter a string 1/0
1
Enter input string:aabb
(q0,a)->(q1,a)->(q1,b)->(q2,b)->(q2)
String accepted
do you want to enter a string 1/0
1
Enter input string:aaaabbbbbbbbc
(q0,a)->(q1,a)->(q1,a)->(q1,b)->(q2,b)->(q2,b)->(q2,b)->(q2,b)->(q2,b)->(q2,b)->(q2,b)->(q2,b)->(q2,c)->invalid
String rejected
do you want to enter a string 1/0
|

```

CONCLUSION

In this experiment we have implemented DFA using C successfully.

Experiment-2

AIM

Implementing a C program to find a regular grammar from Deterministic Finite Automaton (DFA).

Input: The set of states (Q), set of input alphabet (T), set of final states (F) and Transition table.

Output: The grammar $G = (N, T, P, S)$.

ALGORITHM

Finding Regular Grammar from DFA

- Step 1: Read the number of states and input symbols of the DFA.
- Step 2: Read the input alphabet symbols and store them.
- Step 3: Read the DFA transition table for all states and input symbols.
- Step 4: Read the number of final states and mark the final states.
- Step 5: Treat the last state as a dead state and exclude it from grammar generation.
- Step 6: Define the set of non-terminals N using DFA states except the dead state.
- Step 7: Define the set of terminals T using the DFA input alphabet.
- Step 8: Set the start symbol S as the initial state q_0 .
- Step 9: For each DFA transition $q_i \xrightarrow{a} q_j$, generate a production rule $q_i \rightarrow aq_j$
- Step 10: For each final state q_i , add the production rule $q_i \rightarrow \epsilon$.
- Step 11: Output the regular grammar $G = (N, T, P, S)$

SOURCE CODE

```
#include<stdio.h>

int main(){
    int ns,na,nf;
    int i,j;

    printf("Enter number of states:");
    scanf("%d",&ns);

    printf("Enter number of input symbols:");
    scanf("%d",&na);

    char alpha[na];
    printf("Enter input symbols:\n");
    for(i=0;i<na;i++){
        scanf(" %c",&alpha[i]);
    }

    int trans[ns][na];
    printf("Enter transition table (next states):\n");
    for(i=0;i<ns;i++){
        for(j=0;j<na;j++){
            scanf("%d",&trans[i][j]);
        }
    }

    int final[ns];
    for(i=0;i<ns;i++)
```

```

    final[i]=0;

    printf("Enter number of final states:");
    scanf("%d",&nf);

    printf("Enter final states:\n");
    for(i=0;i<nf;i++){
        int x;
        scanf("%d",&x);
        final[x]=1;
    }

    int dead = ns - 1; /* last state is treated as dead state */

    /* OUTPUT GRAMMAR */
    printf("\nRegular Grammar G = (N, T, P, S)\n");

    printf("N = { ");
    for(i=0;i<ns;i++){
        if(i != dead)
            printf("q%d ",i);
    }
    printf("}\n");

    printf("T = { ");
    for(i=0;i<na;i++){
        printf("%c ",alpha[i]);
    }
    printf("}\n");

    printf("S = q0\n");

    printf("P:\n");
    for(i=0;i<ns;i++){
        if(i == dead) continue; /* skip dead state */

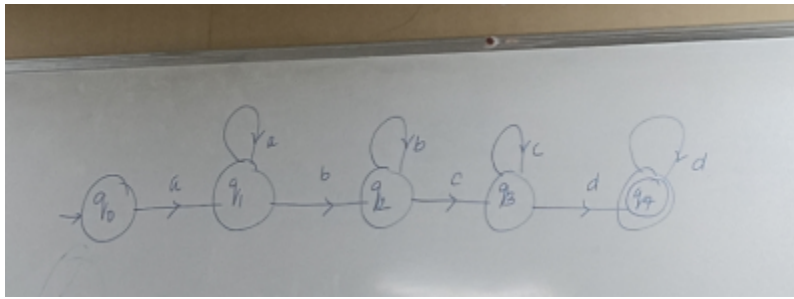
        for(j=0;j<na;j++){
            if(trans[i][j] != dead){
                printf("q%d -> %cq%d\n",i,alpha[j],trans[i][j]);
            }
        }

        if(final[i]){
            printf("q%d -> E\n",i);
        }
    }

    return 0;
}

```

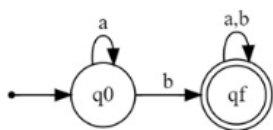
OUTPUT



```
C:\Users\ex1\Desktop\good\c X + v
Enter number of states:6
Enter number of input symbols:4
Enter input symbols:
a
b
c
d
Enter transition table (next states):
1 5 5 5
1 2 5 5
5 2 3 5
5 5 3 4
5 5 5 4
5 5 5 5
Enter number of final states:1
Enter final states:
4

Regular Grammar G = (N, T, P, S)
N = { q0 q1 q2 q3 q4 }
T = { a b c d }
S = q0
P:
q0 -> aq1
q1 -> aq1
q1 -> bq2
q2 -> bq2
q2 -> cq3
q3 -> cq3
q3 -> dq4
q4 -> dq4
q4 -> E

-----
Process exited after 74.35 seconds with return value 0
Press any key to continue . . . |
```




```
C:\Users\ex1\Desktop\good\C X + v
Enter number of states:2
Enter number of input symbols:2
Enter input symbols:
a
b
Enter transition table (next states):
0 1
1 1
Enter number of final states:1
Enter final states:
1

Regular Grammar G = (N, T, P, S)
N = { q0 q1 }
T = { a b }
S = q0
P:
q0 -> aq0
q0 -> bq1
q1 -> aq1
q1 -> bq1
q1 -> e

-----
Process exited after 12.85 seconds with return value 0
Press any key to continue . . . |
```

CONCLUSION

Successfully found the Regular Grammar from DFA in C.

Experiment-3

AIM

Implementing a Deterministic Finite Automaton (DFA) from Non-deterministic Finite Automata (NFA) without ϵ -edges using C language.

ALGORITHM

Conversion of NFA to DFA (Without ϵ -Transitions)

Step 1: Read the number of NFA states, input symbols, and the input alphabet.

Step 2: Read the NFA transition table for each state and input symbol.

Step 3: Read the initial state and the set of NFA final states.

Step 4: Initialize the first DFA state as the set containing the NFA initial state.

Step 5: Apply subset construction to generate new DFA states from existing DFA states.

Step 6: For each DFA state and input symbol, compute the reachable set of NFA states.

Step 7: Add the new DFA state if it is not already present.

Step 8: Store transitions between DFA states for each input symbol.

Step 9: Mark a DFA state as final if it contains at least one NFA final state.

Step 10: Print all DFA states in set representation.

Step 11: Print the DFA transition table with rows and columns as DFA states and entries as input symbols.

SOURCE CODE

```
#include <stdio.h>

#define MAX 20
#define MAX_DFA 50

int n, na;
int i, j, k, x, t, p;

/* NFA[state][symbol][state] */
int nfa[MAX][MAX][MAX];

/* DFA */
int dfaStates[MAX_DFA][MAX];
int dfaTrans[MAX_DFA][MAX];
int dfaFinal[MAX_DFA];
int dfaCount;

/* NFA final states */
int nfaFinal[MAX];

char alpha[MAX];

/* compare two DFA states */
int same(int a[], int b[]){
    for(p = 0; p < n; p++){
        if(a[p] != b[p])
```

```

        return 0;
    }
    return 1;
}

/* print DFA state as set {q0,q1,...} */
void printDFAState(int state[]){
    int first = 1;
    printf("{");
    for(int i = 0; i < n; i++){
        if(state[i]){
            if(!first) printf(",");
            printf("q%d", i);
            first = 0;
        }
    }
    printf("}");
}

/* print DFA states list */
void printDFAStates(){
    printf("\nDFA STATES:\n");
    for(i = 0; i < dfaCount; i++){
        printf("D%d = ", i);
        printDFAState(dfaStates[i]);
        if(dfaFinal[i])
            printf(" (FINAL)");
        printf("\n");
    }
}

/* print transition table: States × States, cell = input symbol */
void printDFATransitionTable(){
    printf("\nDFA TRANSITION TABLE (States × States)\n\n");

    /* column headers */
    printf("\t");
    for(j = 0; j < dfaCount; j++){
        printf("D%d\t", j);
    }
    printf("\n");

    /* rows */

```

```

for(i = 0; i < dfaCount; i++){
    printf("D%d\t", i);

    for(j = 0; j < dfaCount; j++){
        char symbol = '-';

        for(k = 0; k < na; k++){
            if(dfaTrans[i][k] == j){
                symbol = alpha[k];
                break;
            }
        }

        printf("%c\t", symbol);
    }
    printf("\n");
}

int main(){

    int initial, nf;
    int found;
    int newState[MAX];

    printf("Enter number of NFA states: ");
    scanf("%d", &n);

    printf("Enter number of input symbols: ");
    scanf("%d", &na);

    printf("Enter input symbols:\n");
    for(i = 0; i < na; i++){
        scanf(" %c", &alpha[i]);
    }

    printf("Enter NFA transition tables (0 or 1):\n");
    for(j = 0; j < na; j++){
        printf("For symbol %c:\n", alpha[j]);
        for(i = 0; i < n; i++){
            for(k = 0; k < n; k++){
                scanf("%d", &nfa[i][j][k]);
            }
        }
    }
}

```

```

    }
}

printf("Enter initial state: ");
scanf("%d", &initial);

printf("Enter number of NFA final states: ");
scanf("%d", &nf);

for(i = 0; i < n; i++)
    nfaFinal[i] = 0;

printf("Enter NFA final states:\n");
for(i = 0; i < nf; i++){
    scanf("%d", &x);
    nfaFinal[x] = 1;
}

/* initialize DFA states */
for(i = 0; i < MAX_DFA; i++){
    for(j = 0; j < n; j++){
        dfaStates[i][j] = 0;
    }
}

dfaStates[0][initial] = 1;
dfaCount = 1;

/* subset construction */
for(i = 0; i < dfaCount; i++){
    for(j = 0; j < na; j++){

        for(k = 0; k < n; k++)
            newState[k] = 0;

        for(k = 0; k < n; k++){
            if(dfaStates[i][k]){
                for(x = 0; x < n; x++){
                    if(nfa[k][j][x])
                        newState[x] = 1;
                }
            }
        }
    }
}

```

```

        found = -1;
        for(t = 0; t < dfaCount; t++){
            if(same(newState, dfaStates[t])){
                found = t;
                break;
            }
        }

        if(found == -1){
            found = dfaCount;
            for(k = 0; k < n; k++){
                dfaStates[dfaCount][k] = newState[k];
            }
            dfaCount++;
        }

        dfaTrans[i][j] = found;
    }
}

/* identify DFA final states */
for(i = 0; i < dfaCount; i++){
    dfaFinal[i] = 0;
    for(j = 0; j < n; j++){
        if(dfaStates[i][j] && nfaFinal[j]){
            dfaFinal[i] = 1;
            break;
        }
    }
}

/* OUTPUT */
printDFASStates();
printDFATransitionTable();

return 0;
}

```

OUTPUT

```

D2      a      -      -
gg@ubuntu:~/Desktop$ gcc sub.c -o gcc
gg@ubuntu:~/Desktop$ ./gcc
Enter number of NFA states: 6
Enter number of input symbols: 1
Enter input symbols:
a
Enter NFA transition tables (0 or 1):
For symbol a:
0 1 0 0 1 0
0 0 1 0 0 0
0 0 0 1 0 0
0 0 0 0 0 0
0 0 0 0 0 1
0 0 0 0 1 0
Enter initial state: 0
Enter number of NFA final states: 2
Enter NFA final states:
3
5

```

```

DFA STATES:
D0 = {q0}
D1 = {q1,q4}
D2 = {q2,q5} (FINAL)
D3 = {q3,q4} (FINAL)
D4 = {q5} (FINAL)
D5 = {q4}

```

DFA TRANSITION TABLE (States × States)

	D0	D1	D2	D3	D4	D5
D0	-	a	-	-	-	-
D1	-	-	a	-	-	-
D2	-	-	-	a	-	-
D3	-	-	-	-	a	-
D4	-	-	-	-	-	a
D5	-	-	-	-	a	-

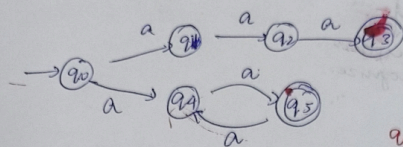
```

gg@ubuntu:~/Desktop$ █

```

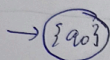
Implementing DFA from NFA without ϵ edges.
 38) NFA $\rightarrow$ DFA

$$L = \{ a^{2n} \mid n \geq 1 \} \cup \{ a^{33} \}$$



$$\delta(q_0, a) = \{q_1, q_4\}$$

initial q_0 .

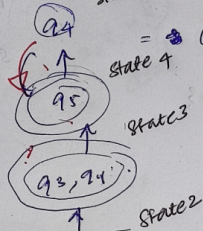


	q_0	q_1	q_2	q_3	q_4	q_5
q_0	-	a	-	-	a	-
q_1	-	-	a	-	-	-
q_2	-	-	-	a	-	-
q_3	-	-	-	-	-	a
q_4	-	-	-	-	-	a
q_5	-	-	-	-	a	-

transition for req. DFA.

$$\delta'(\{q_1, q_m, \dots, q_n\}, a) = \delta(q_1, a) \cup \delta(q_m, a) \cup \dots \cup \delta(q_n, a)$$

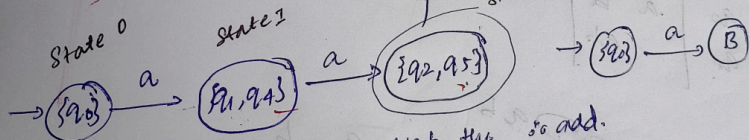
A set.



$$= \{q_2, q_3, \dots, q_k\}$$

B new set

check this is DFA.
already exist else NO



$$\delta'(\{q_0\}, a) = \{q_1, q_4\} \rightarrow \text{Not the } \epsilon \text{ add.}$$

$$\delta'(\{q_1, q_4\}, a) = \delta(q_1, a) \cup \delta(q_4, a) = q_2 \cup q_5$$

$$\{q_2, q_5\} \rightarrow \text{Not there } \epsilon \text{ add}$$

$$\delta'(\{q_2, q_5\}, a) = \delta(q_2, a) \cup \delta(q_5, a) = \{q_3, q_4\}$$

$$\delta'(\{q_3, q_4\}, a) = \delta(q_3, a) \cup \delta(q_4, a) = \{q_5\}$$

$$\delta'(\{q_5\}, a) = \{q_4\}$$

$$\delta'(\{q_4\}, a) = \{q_5\}$$

CONCLUSION

Successfully converted the NFA to DFA (Without ϵ -Transitions)

Experiment-4

AIM

- a) To Implement a DFA in LEX code which accepts Odd number of a's and even number of b's followed by ccc.
- b) To implement a DFA in LEX code which accepts strings over {a, b, c} having bca as substring.

a)

ALGORITHM a)

DFA to Accept Odd number of a's, Even number of b's followed by ccc

Step 1: Start in the initial state and read the input string character by character.

Step 2: Move between states S2 and S3 to track odd and even occurrences of character a.

Step 3: Allow transition to the b-processing state only after an odd number of a is detected.

Step 4: Move between states S4 and S5 to track odd and even occurrences of character b.

Step 5: Allow transition to the c-processing state only after an even number of b is detected.

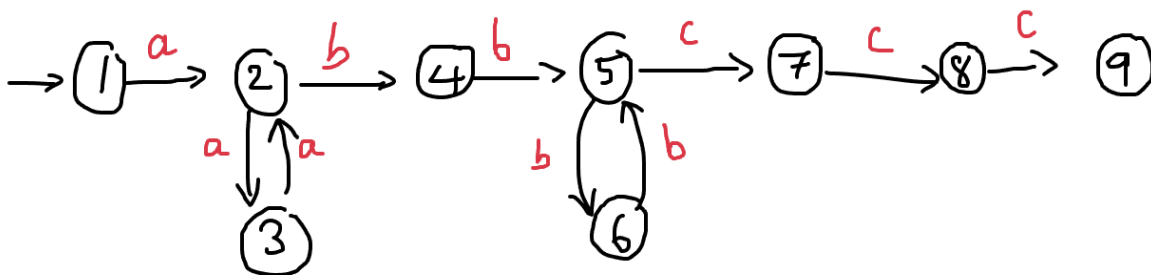
Step 6: Read exactly three consecutive c characters by transitioning through states S7, S8, and S9.

Step 7: If the input ends immediately after the third c, accept the string.

Step 8: If any invalid character or incorrect sequence occurs, move to the DEAD state.

Step 9: In the DEAD state, consume the remaining input and reject the string.

Step 10: Reset to the initial state and wait for the next input string.



(final state is 9)

SOURCE CODE

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

/* %x makes states exclusive to prevent default printing of characters */
%x S2 S3 S4 S5 S7 S8 S9 DEAD
```

%%

```
<INITIAL>a      { BEGIN S2; }
<INITIAL>\n     { printf("Enter string: "); }
<INITIAL>.      { BEGIN DEAD; }
```

/* Transitions for 'a' (States 2 and 3) */

```
<S2>a          { BEGIN S3; }
<S3>a          { BEGIN S2; }
<S2>b          { BEGIN S4; } /* Move to 'b' only from odd 'a' */
<S2,S3>.\n     { BEGIN DEAD; }
```

/* Transitions for 'b' (States 4 and 5) */

```
<S4>b          { BEGIN S5; }
<S5>b          { BEGIN S4; }
<S5>c          { BEGIN S7; } /* Move to 'c' only from even 'b' */
<S4,S5>.\n     { BEGIN DEAD; }
```

/* Transitions for 'ccc' (States 7, 8, and 9) */

```
<S7>c          { BEGIN S8; }
<S8>c          { BEGIN S9; }
<S9>c          { /* If you want exactly 3 c's, staying here might be a rejection depending on lab rules
*/
                BEGIN DEAD;
            }
<S9>\n         {
                printf("\nRESULT: ACCEPTED\n\n");
                printf("Enter string: ");
                BEGIN INITIAL;
            }
<S7,S8,S9>.    { BEGIN DEAD; }
<S7,S8>\n       { BEGIN DEAD; }
```

/* DEAD state handles all invalid sequences and prevents repetition */

```
<DEAD>[^\\n]* { /* Consume the rest of the invalid string */ }
<DEAD>\n       {
                printf("\nRESULT: REJECTED\n\n");
                printf("Enter string: ");
                BEGIN INITIAL;
            }
}
```

%%

```

int main() {
    printf("--- DFA: (odd a)(even b)ccc ---\n");
    printf("Enter string: ");
    yylex();
    return 0;
}

int yywrap() {
    return 1;
}

```

OUTPUT

```

Enter string: ^C
gg@ubuntu:~/Desktop$ flex 1.l
gg@ubuntu:~/Desktop$ gcc lex.yy.c -o 1
gg@ubuntu:~/Desktop$ ./1
DFA Machine (odd a)(even b)ccc
Enter string: accc

RESULT: REJECTED

Enter string: abbccc

RESULT: ACCEPTED

Enter string: abaccc

RESULT: REJECTED

Enter string: faogknsl

RESULT: REJECTED

Enter string:

```

b)

ALGORITHM b)

DFA to Accept Strings Containing Substring bca

Step 1: Start from the initial state and scan the input string character by character.

Step 2: Ignore characters a and c until the character b is encountered.

Step 3: On reading b, move to the next state to check for the character c.

Step 4: If c is found after b, move to the next state to check for the character a.

Step 5: If a is found after bc, move to the accepting state indicating substring bca is detected.

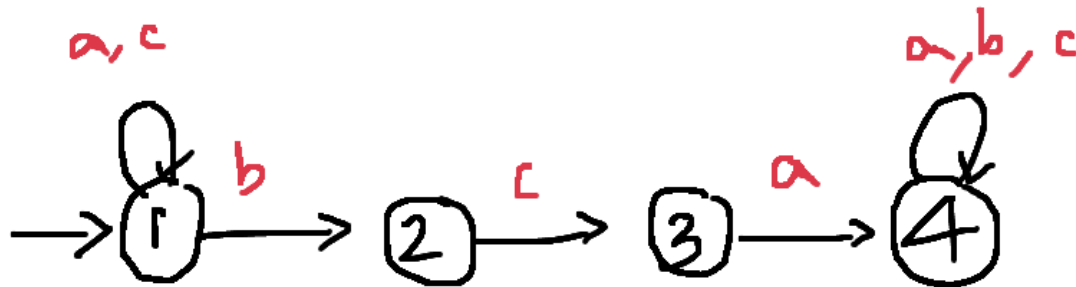
Step 6: Once in the accepting state, allow any combination of a, b, or c.

Step 7: Accept the string if the input ends while in the accepting state.

Step 8: If any invalid character or wrong sequence occurs, move to the DEAD state.

Step 9: In the DEAD state, consume the remaining input and reject the string.

Step 10: After acceptance or rejection, reset to the initial state for the next input.



SOURCE CODE

```
%{
#include <stdio.h>
#include <stdlib.h>
%}

/* %x makes states exclusive to prevent the default echo behavior */
%x S2 S3 S4 DEAD

%%

<INITIAL>b      { BEGIN S2; }
<INITIAL>[ac]   { BEGIN INITIAL; }
<INITIAL>\n     { printf("\nRESULT: REJECTED\n\nEnter string: "); }
<INITIAL>.      { BEGIN DEAD; }

<S2>c           { BEGIN S3; }
<S2>b           { BEGIN S2; }
<S2>a           { BEGIN INITIAL; }
<S2>\n          { printf("\nRESULT: REJECTED\n\nEnter string: "); BEGIN INITIAL; }
<S2>.           { BEGIN DEAD; }

<S3>a           { BEGIN S4; }
<S3>b           { BEGIN S2; }
<S3>c           { BEGIN INITIAL; }
<S3>\n          { printf("\nRESULT: REJECTED\n\nEnter string: "); BEGIN INITIAL; }
<S3>.           { BEGIN DEAD; }
```

```

<S4>[abc]      { BEGIN S4; }
<S4>\n         {
                printf("\nRESULT: ACCEPTED\n\nEnter string: ");
                BEGIN INITIAL;
            }
<S4>.          { BEGIN DEAD; }

<DEAD>[^\\n]+   { /* Do nothing - swallow rest of invalid string */ }
<DEAD>\n        {
                printf("\nRESULT: REJECTED\n\nEnter string: ");
                BEGIN INITIAL;
            }

%%

int main() {
    printf("--- Substring 'bca' Evaluator ---\n");
    printf("Enter string: ");
    yylex();
    return 0;
}

int yywrap() {
    return 1;
}

```

OUTPUT

```
gg@ubuntu:~/Desktop$ flex 2.1
gg@ubuntu:~/Desktop$ gcc lex.yy.c -o 2
gg@ubuntu:~/Desktop$ ./2
substring 'bca' {a,b,c}
Enter string: bca

RESULT: ACCEPTED

Enter string: abca

RESULT: ACCEPTED

Enter string: bac

RESULT: REJECTED

Enter string: nwk

RESULT: REJECTED

Enter string: hilleo

RESULT: REJECTED

Enter string: aaabbcabcacaacac

RESULT: ACCEPTED
```

CONCLUSION

The given DFA was successfully implemented using LEX and its correctness was verified through valid and invalid test strings.